Evaluating the Effect of Domain Adaptation & Debiasing on Breast Tumour Classification

Saatvik Ramachandra Kambhampati (220261740)

A technical report presented for the
BEng in Electronics & Computer Engineering (H652)
2025



Department of Electrical & Electronics Engineering
School of Engineering
Newcastle University
United Kingdom
2nd May 2025

(Student) Saatvik Kambhampati
email: s.kambhampati2@newcastle.ac.uk

(1st Supervisor) Kabita Adhikari
email: kabita.adhikari@newcastle.ac.uk

(2nd Supervisor) Shidong Wang
email: shidong.wang@newcastle.ac.uk

October 17, 2026

Abstract

According to Cancer Research UK, breast cancer claims the lives of 56,822 individuals, in the UK, each year. While machine learning has shown promise in breast tumour classification, gaps remain in addressing data bias and model generalisability across datasets. This project investigates the effectiveness of debiasing and domain adaptation techniques in improving classification performance. Using the Wisconsin (Breast Cancer) Diagnostic dataset (569 instances) and the Woods' Mammography Dataset (11,183 instances), six models were tested for baseline performance, transfer learning, and sensitivity to bias. Of the six models, five are classical and one is deep learning. Debiasing methods like ADASYN and SMOTE were applied, and CORAL was used for domain adaptation. Deep Neural Networks performed best at baseline; ADASYN achieved high sensitivity but a big accuracy drop, while SMOTE offered a better balance. CORAL improved performance on malignant cases, raising specificity up to 35% (from close to 0%) in some cases. These findings highlight the need for bias-aware and adaptable models in medical AI, showing practical paths toward more robust tumour classification systems.

Contents

1 Introduction **1**
1.1 Breast Cancer . . . 1
1.2 Breast Tumour Classification: Current Method & Use of AI . . . 1

2 Literature Review **2**
2.1 Overview of Report and Aim of Project . . . 2

3 Breast Cancer Datasets & Data Pre-Processing **3**
3.0.1 Diversity, Equality, and Inclusion . . . 3
3.1 Data Pipeline for Data Pre-Processing . . . 3
3.1.1 Data Cleaning & Formatting . . . 4
3.2 Feature Selection (FS) . . . 4
3.3 Debiasing Techniques . . . 5
3.3.1 Random Over- and Under- Sampling Techniques . . . 5
3.3.2 Synthetic Minority Over-sampling TEchnique (SMOTE) . . . 5
3.3.3 Adaptive Synthetic Sampling (ADASYN) . . . 7
3.4 Transfer Learning & Domain Adaptation . . . 8
3.4.1 The CORrelation ALignment (CORAL) Technique . . . 8

4 Machine Learning Algorithms **10**
4.1 Initial Implementation Development and Model Hyperparameters . . . 11
4.1.1 Hyperparameters . . . 11

5 Experiments and Methodology **12**
5.1 Optimising Feature Selection . . . 12
5.2 Baseline Breast Tumour Classification with ML . . . 13
5.3 Selecting Debiasing Techniques . . . 13
5.4 Testing Domain Adaptation for Transfer Learning . . . 13
5.5 Performance Metrics . . . 14

6 Results **15**
6.1 Optimising Feature Selection - Results . . . 15
6.2 Baseline Classification With ML - Results . . . 16
6.3 Selecting Debiasing Techniques (With Mammography Data) - Results . . . 17
6.4 Testing Domain Adaptation - Results . . . 20

7 Analysis & Discussion of Results **22**

8 Conclusion & Evaluation **24**
8.1 Conclusion . . . 24
8.2 Future Work . . . 24
8.3 Project Evaluation . . . 24
8.3.1 Personal Development & Self-Reflection . . . 25

A Appendix A: Algorithms, Definitions, and Maths **29**
A.1 Correlation Feature Selection . . . 29
A.1.1 CFS Algorithm . . . 30
A.2 ADASYN Debiasing Algorithm . . . 33
A.3 Other Definitions . . . 33

B Appendix B: Code Implementation **34**
B.1 Information . . . 34
B.2 Model Clases . . . 34

List of Figures

1 Diagram of a breast . . . 1
2 Shows the data flow pipeline of the data preprocessing carried out. . . . 4
3 Visual explanations of (a) oversampling and (b) undersampling. . . . 5
4 Diagrams of various machine learning models. . . . 10
5 UML Class Diagram of the Models. Note: `torch.nn.Module` is the PyTorch DNN class, and `pd.DF` is the Panda's DataFrame class. **The full code can be found in Appendix 2.** . . . 11
6 Number of features vs accuracies for all models (WBCD Dataset) . . . 15
7 Performance of each model on the WBCD Dataset . . . 16
8 Performance of each model on the Mammography dataset (n.b. no debiasing involved) . . . 16
9 Performance of models on mammography dataset with synthesis-based debiasing. 18
10 Performance of models on mammography dataset with random-based debiasing. 19
11 Performance of WBCD-trained models on mammography set. . . . 20

List of Tables

1 Summary of Breast Cancer Datasets . . . 3
2 Key hyper-parameters used in this study . . . 12
3 Accuracy (%) . . . 17
4 Sensitivity (%) . . . 17

1 Introduction

Cancer [1] occurs when abnormal cells (tumours) divide and duplicate in uncontrollable ways [1]. Only 50% of patients survive for 10 or more years [2]. Tumours come in two forms: benign and malignant. Benign or non-cancerous tumours are slow-growing and do not spread easily to other body areas or tissues. Malignant, or cancerous, tumours, grow rapidly, have irregular borders and often invade surrounding tissue and spread to other body areas using a process known as 'metastasis' [3]. It is important to identify these tumours so appropriate treatment can be started immediately.

1.1 Breast Cancer

Breast tumours can originate from the inner lining of the milk ducts or the 'lobules that supply [these] ducts with milk' [4] (Figure 1). They cause the appearance of lumps, change in the size and shape of the breast, or armpit swellings [5]. Breast cancer is the most common cause of cancer in women and the second most common cause of cancer deaths in the United States [4].

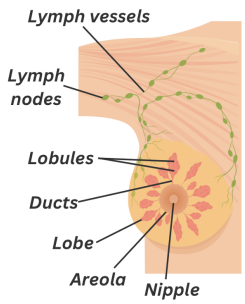


Figure 1: Diagram of a breast. [6]

Breast cancer has therefore been widely studied, with availability of large datasets consisting of observational data, clinical measurements, and images. Apart from being a crucial area of clinical research, it also makes for a useful case study for research on tumour classification using Artificial Intelligence (AI).

1.2 Breast Tumour Classification: Current Method & Use of AI

Typically, classification is done by taking a biopsy sample of the tumour and a pathologist examining it microscopically [7]. This can be very time consuming and inefficient given the number of patients and the availability of expert pathologists. This might lead to a delayed diagnosis of malignant cases causing delayed treatment. Automation can aid greatly in reducing the diagnosis/classification time, enabling earlier and timely treatment.

There has been an increasing use of AI in healthcare, including to diagnose COVID-19 [8], with potential to improve both accuracy and efficiency (by reducing manual work) [9]. Given that AI has historically performed well in other medical classification problems and the need for automating breast tumour classification, there is great scope for it's application here.

2 Literature Review

Various models and techniques have been implemented for breast cancer classification. Among the most common are Support Vector Machines (SVMs), Decision Trees (DTCs), and k Nearest Neighbours ($k-$NNs). Researchers have tested various SVM kernels and have shown accuracies of up to 98.82% [10] when combined with Linear Discriminant Analysis (LDA). With ensemble models of various kernels, this accuracy can further be pushed up to 99.29% [11]. There is also a lot of literature about the use of $k-$NNs, DTCs, and Random Forests (RFCs). It has been found that lower k values yield higher accuracies for $k-$NNs (exceeding 96%) [12] [13]. Naive Bayes Classifiers (NBC) had some of the lowest accuracies, with a range of 65 [14] to 96.19% [15], which increased to 98.24% when combined with LDA [10].

Conventional Artificial Neural Networks (ANNs) and advanced variants like Deep Belief Networks (DBN) have achieved impressive accuracy rates of about 99% [16]. Using a mammography set, various convolution networks, such as GoogleNet, VGCNet, and ResNet, were used in tandem with a fully connected classification layer to obtain an accuracy of up to 97% (using average pooling classification) n [17]. A VGG convolution network has been used and various features extracted from each layer of different blocks in order to classify tumours [18].

Although there is some literature available on debiasing, it is predominantly focused on using imaging datasets. There are very few that investigate domain adaptation techniques, especially when looking at numerical datasets. Therefore, it is important to investigate the effect of both debiasing techniques and domain adaptation techniques on breast tumour classification based on conventional datasets.

2.1 Overview of Report and Aim of Project

This project aims to provide diagnostic tools in the classification of breast tumours. Compared to traditional methods, machine learning (ML) can drastically cut down the classification time of cancer. The project also aims to determine whether feature selection (FS), debiasing and domain adaptation (DA) methods can enable ML models to better classify breast tumours.

3 Breast Cancer Datasets & Data Pre-Processing

Of the various datasets available for breast tumour analysis, two are used here as their features describe similar physical properties (or dimensions) of breast tumours.

Table 1 provides information about both datasets used in this project.

Table 1: Summary of Breast Cancer Datasets

Dataset	Short-Hand	Class Bias	# Features	Distribution Notes	Example Features	Reference (Year)
Wisconsin (Diagnostic)	WBCD	63% benign / 37% malignant	30	6 approx. normal distribution; 24 skewed to higher values	Mean radius, texture, perimeter, smoothness, compactness, symmetry, etc.	[19] (1995)
Woods' Mammography	Mammography dataset	98% benign / 2% malignant	6	–	Area, grey level, gradient strength	[20] (1993)

For both datasets, the patients were completely anonymised. Each patient (instance) had a unique ID and the characteristics of their tumour, however no personal information such as age, sex, name, or occupation was used. Furthermore, the data was secured against accidental loss by having numerous back-ups of the original, including saving the data after the different stages of preprocessing.

3.0.1 Diversity, Equality, and Inclusion

Patient demographics in each dataset remains unknown, which could lead to the reliability of the models suffering as tumour traits and malignancy risk can differ by ethnicity and sex.

3.1 Data Pipeline for Data Pre-Processing

Data pre-processing is concerned with removing unwanted variability from the dataset so that the useful information related to the objective can be used for efficient modelling of the solution [21]. It is used to estimate, or remove instances with, missing values. Estimation can be done using data imputation approaches [21]. **Figure 2** shows the data pipeline for the pre-processing of the datasets. In this project, feature selection, debiasing, and domain adaptation are particularly looked at.

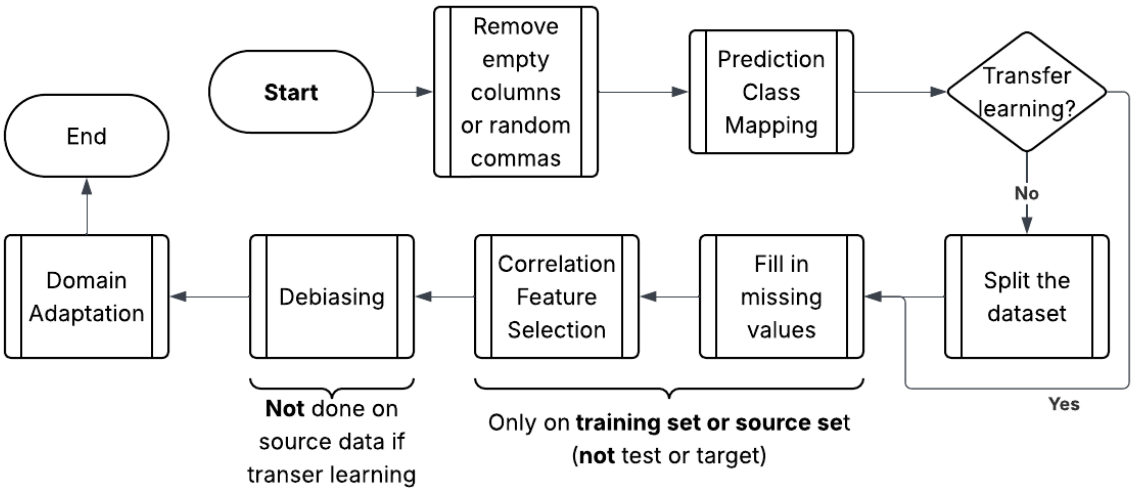


Figure 2: Shows the data flow pipeline of the data preprocessing carried out.

3.1.1 Data Cleaning & Formatting

The data was cleaned by removing empty columns and rows. Missing values in the instances were dealt with using $k-$NN-based imputation. Where a value is missing, a synthetic value is substituted by observing the value of the nearest neighbours in the same feature [22]. This was done on both the training and testing datasets, by using the distribution of the training data to impute the testing data [22].

3.2 Feature Selection (FS)

The FS algorithm chosen for this project was the Correlation Feature Selection (CFS) method. CFS works by using a value known as the Pearson's Correlation Coefficient (PCC) [23] to compute the correlation between every distinct permutation of features. Algorithmically, the PCC between two variables X and Y, $r_{X,Y}$, is denoted as $r_{X,Y} \leftarrow \mathbf{PCC}(X, Y)$. CFS' underlying principle is based on categorising any given feature as *relevant* or *redundant*.

Relevance measures how correlated the feature is with the predictive class. *Redundancy*, however, evaluates a feature's correlation with another feature (**Definitions A.2 & A.3** (**Appendix A.1**)). CFS works by choosing the p most useful features (those that are both *relevant* and *not redundant* (**Definition A.4**, **Appendix A.1**).

The full algorithm (**Algorithm 5**) is in **Appendix A.1.1**. The formal definition of a 'dataset' can be found as **Definition A.1** in **Appendix A**.

3.3 Debiasing Techniques

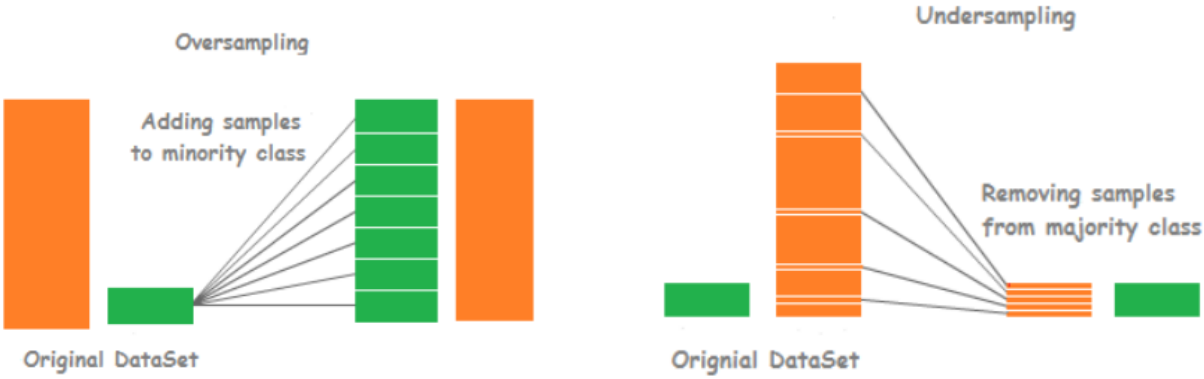
An important challenge of clinical datasets is the lack of balance between classes. Frequently, the class of no disease overwhelms that of the disease, as exemplified in the Mammography dataset. When fitting models, this means that the minority class is "more difficult to learn" compared to the majority class[24]. Fitted models often predict unseen data as the majority class due to greater exposure to examples that fit those patterns. Debiasing works to remedy this issue by adding or removing instances to reduce the difference between the number of instances per class.

The performance of the algorithms explained here will be tested in **Chapter 5.3**.

3.3.1 Random Over- and Under- Sampling Techniques

Random oversampling is the *duplication* of random instances of the *minority* class [25], **Figure 3a**. It suffers from limitations, such as an increased risk of over-fitting (due to the same instance existing multiple times), and an increase in computational intensity of training due to the set size increasing [25]. The latter is less of an issue for smaller datasets and hence oversampling is more useful there [25].

Undersampling is concerned with *removing* random instances of the *majority* class [25], **Figure 3b**. The challenge is that model accuracy may be affected as important data may get deleted [25]. This is not an issue for larger datasets and hence undersampling is more useful there [25].



(a) Oversampling [26]

(b) Undersampling [26]

Figure 3: Visual explanations of (a) oversampling and (b) undersampling.

3.3.2 Synthetic Minority Over-sampling TEchnique (SMOTE)

SMOTE is a type of minority class over-sampling technique that utilises interpolation to artificially create minority class instances [27].

> "The minority class is over-sampled by taking each minority class [instance] and introducing synthetic [instances] along the line segments joining any/all of the k minority class nearest neighbors [27]."

Below, the typical SMOTE algorithm (along with a **Populate** algorithm) [27] is examined.

Algorithm 1 Populate [27]

Require: The "amount" of SMOTE $N \in \mathbb{R}\%$, some instance $T_i \in T$ and its k-nearest neighbours' indexes neighbours_i.

Ensure: A set S of synthetic samples neighbouring T_i. $|T_{\text{syn}}| = \frac{N}{100} \cdot |T|$

Let p be the amount of features in T.
$a \leftarrow 0$
while $N \neq 0$ **do**
    Let $n_n \in neighbour_i$ be the index of some nearest neighbour of T_i s.th. $0 \leq n_n \leq k$.
    $n_n \leftarrow \textbf{rand}(1, k+1)$
    $g \leftarrow \text{neighbours}_i[n_n]$

    for $j \in \mathbb{N}_0$ **from** $j = 0$ **to** $j = p$ **do**
        Let $t_{g,j}$ be T_g's observation for the j-th feature.
        $\Delta \leftarrow t_{g,j} - t_{i,j}$
        $\gamma \leftarrow \textbf{rand}(0, 1)$
        Let $s_{a,b}$ be S_a's observation for the b-th feature.
        $s_{a,j} \leftarrow t_{i,j} + \gamma \cdot \Delta$
    end for
    $S \leftarrow S \cup S_a$
    $a \leftarrow a + 1$
    $N \leftarrow N + 1$
end while
return S

The **Populate** algorithm is responsible for actually *creating* the synthetic instances using some sample instance T_i (from the set of minority instances T) and its $k-$NNs' indices in the dataset. The other input, N, represents the amount of instances that should be synthesised near T_i.

Algorithm 2 shows the SMOTE algorithm, which uses **Algorithm 1** for the creation of N synthetic instances by interpolating between the instance and one of its randomly chosen neighbours.

Algorithm 2 SMOTE - [27]

Require: A set of minority-classified instances T s.th. $T \subseteq D$ (as in **Definition A.1**), the "amount" of SMOTE to be carried out $N \in \mathbb{R}\%$, and a number $k \in \mathbb{N}$ of nearest neighbours.

Ensure: A set T_{syn} of synthetic samples to be added to D. $|T_{\text{syn}}| = \frac{N}{100} \cdot |T|$

if $N < 100$ **then**
    Randomise the minority instances.
    Then only use N% of the minority instances.
    $T \leftarrow \frac{N}{100} \cdot T$
    $N \leftarrow 100$
end if
$N \leftarrow \lfloor \frac{N}{100} \rfloor$
Let p be the amount of features in T.

for $i \in \mathbb{N}_0$ **from** $i = 0$ **to** $i = |T| - 1$ **do**
    *Let **KNN**$(k,\ T,\ T_i)$ denote the function for a k-Nearest Neighbours algorithm.*
    Let neighbours$_i$ be the k-length row vector of the indices (in T) k-nearest neighbours to T_i s.th. $T_n \subseteq T$, where $n \in$ neighbours$_i$.
    neighbours$_i \leftarrow$ **KNN**$(k,\ T,\ T_i)$
    $S \leftarrow$ **Populate**$(N, T_i,$ neighbours$_i)$ *(see **Algorithm 2**.)*
    $T_{\text{syn}_i} \in T \leftarrow S$
end for
return T_{syn}

3.3.3 Adaptive Synthetic Sampling (ADASYN)

ADASYN is a variation of SMOTE that uses a scaling factor $\hat{r}_i$ to determine how many synthetic instances are created for each minority class instance [24]. ADASYN's adaptive algorithm essentially provides SMOTE with information about the number of synthetic instances that should be made. ADASYN can provide any model with balanced data, allowing it to learn the patterns of minority-class instances more easily [24]. A previously designed algorithm [24] is in **Appendix A.2 (Algorithm 6)**

ADASYN works by using $\hat{r}_i$ to compute the number of synthesised instances g_i, and then applying the SMOTE algorithm. The number of synthesised instances can be varied specifically for the particular biased dataset. In theory, this should provide those instances which have a larger number of majority-class instances neighbouring them with more newly synthesised minority instances.

3.4 Transfer Learning & Domain Adaptation

In this chapter, the concept of transfer learning, and problems arising from it, are explained. Domain adaptation and the CORAL approach are introduced in more detail in order to develop the 'inverse CORAL' implementation.

But first, the concept of a domain must be defined.

Definition 3.1 (**Domains** [28])**.**
A *domain* $\mathcal{D}$ is the tuple of a feature space and a marginal distribution

$$(\mathcal{X}, P(X))$$

where X denotes the set of all instances (all X_i (**Definition A.1.2**) s.th. $X_i \subset \mathcal{X}$).

Transfer Learning (TL) is a problem concerned with generalising a model's training *across various domains* [28]. That is, given one dataset that the model is trained on, how well does it perform when tested on another dataset describing the same problem? Typically, the training dataset is the 'source domain' ($\mathcal{D}_S$) dataset and the testing is the 'target domain' ($\mathcal{D}_T$ dataset.

Two datasets may have different domains, $\mathcal{D}_1$, $\mathcal{D}_2$. In such cases, the technical constraints (each model can typically only apply its training on data of a similar domain) of a learning model makes transfer learning difficult. To combat this, Domain Adaptation techniques can be employed.

Definition 3.2 (**Domain Adaptation** - [29] [30])**.**
Given two non-equal domains $\mathcal{D}_1, \mathcal{D}_2$, if $\exists$ some transformation A s.th.

$$A : \mathcal{D}_1 \rightarrow \mathcal{D}_2$$

then it is said that A *adapts* or *encodes* $\mathcal{D}_1$ to $\mathcal{D}_2$.

3.4.1 The CORrelation ALignment (CORAL) Technique

The CORAL domain adaptation technique [30] involves using covariances to align the domains of two datasets. This is done by finding the transformation $\hat{A}$ such that $\hat{A}$ minimises the difference ($||.||_F^2$) between the covariances of the two datasets. This provides the definition of the most optimal transformation.

The ideal case is where $||.||_F^2$ is as close to zero as possible. This then becomes an optimisation problem (OP) where the 'best' encoded dataset can be found.

Definition 3.3 (**Best Encoded Training Dataset**)**.**
Given two datasets in different domains $X_1 \in \mathcal{D}_1$ and $X_2 \in \mathcal{D}_2$, the **best** encoded training dataset $\hat{X}_1 = \hat{A}(X_1)$ is computed by finding the linear transformation $\hat{A}$ that minimises $||.||_F^2$:

$$\hat{A} = \min_{\hat{A}} ||C_1 - C_2||_F^2$$

where is the C_N is the covariance matrix of the dataset X_N and F is the Frobenius Norm (used as a matrix distance metric).

This is an OP, which must be solved by applying singular value decomposition (SVD), applied to C_1:

$$\hat{C_1} = U_{Y_{[1:r]}} \Sigma_{Y_{[1:r]}} V^{\top}_{Y_{[1:r]}} \tag{1}$$

By **Theorem 1** [30, Theorem 1], the solution to the OP in **Definition 3.3** is:

$$\hat{A} = (U_1 \Sigma_1^{+\frac{1}{2}} U_1^{\top}) \cdot (U_{Y_{[1:r]}} \Sigma_{Y_{[1:r]}} V^{\top}_{Y_{[1:r]}}),\ r = \min(r_{C_1}, r_{C_2}) \tag{2}$$

Where $r_{\mathbf{D}} = \text{rank}(\mathbf{D})$.

If $\text{rank}(C_1) \geq \text{rank}(C_2)$ then the minimum norm is zero ($||C_1^{\text{optimal}} - C_2||_F^2 = 0$). Else, if $\text{rank}(C_2) > \text{rank}(C_1)$, the minimum is non-zero.

The solution provided by **Theorem 1** [30, Theorem 1] can be interpreted as follows:

- The first term, $(U_1 \Sigma_1^{+\frac{1}{2}} U_1^{\top})$, 'whitens' X_1 (by essentially decorrelating it [30]).
- The second term, $(U_{Y_{[1:r]}} \Sigma_{Y_{[1:r]}} V^{\top}_{Y_{[1:r]}})$, then 're-colours' it ("re-correlating the [X_1's domain] to [X_2's domain]" [30]).

The actual algorithmic implementation, however, does not rely on SVD. This is because SVD implementations can be inefficient and unstable [30]. Therefore, CORAL is commonly implemented using traditional whitening and re-colouring techniques [30]. This is done by adding a regularisation parameter λ when finding the covariance of the two datasets.

Algorithm 3 CORAL - [30]

Require: Two **Definition A.1**-type datasets $D_1 \in \mathcal{D}_1$ and $D_2 \in \mathcal{D}_2$, and a regularisation parameter $\lambda \in [0, 1]$.

Ensure: An encoded source dataset $\hat{D}_1$.

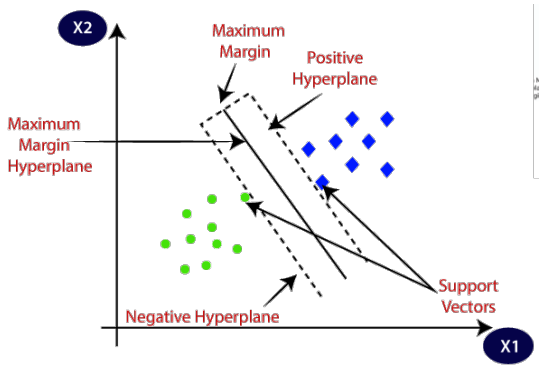
1: *Let I_p be the p by p identity matrix (where p is the number of features).*
2: $C_1 \leftarrow \mathbf{cov}(D_1) + \lambda I_p$
3: $C_2 \leftarrow \mathbf{cov}(D_2) + \lambda I_p$
4: $D_{1_{\text{temp}}} \leftarrow D_1 \cdot C_1^{-\frac{1}{2}}$
5: $\hat{D}_1 \leftarrow D_{1_{\text{temp}}} \cdot C_2^{\frac{1}{2}}$
6: **return** $\hat{D}_1$

Lines 1 – 4 whitens the source data and **line 5** re-colours it.

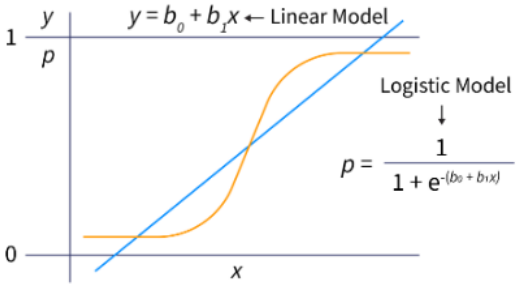
Typically, the CORAL algorithm is used to align the source dataset D_S *to* the target dataset D_T (i.e. $\mathbf{CORAL}(D_S, D_T, \lambda)$). However, this means that the model(s) must be re-trained whenever new data is to be fed into it. If, instead, the D_T is encoded to D_S's domain ($\mathbf{CORAL}(D_T, D_S, \lambda)$), the model will *not* have to be re-trained constantly and will effectively boost the model's generalisation ability. In this report, this is called **'Reverse CORAL'** or 'R-CORAL'.

4 Machine Learning Algorithms

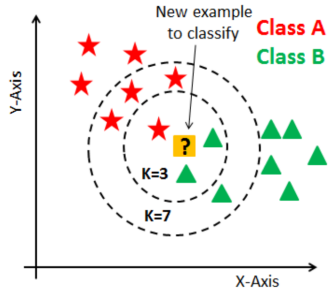
The algorithms chosen for this project (**Figure 4**) are the Support Vector Machine, linear kernel, (SVM-L), Logistic Regression Classifier (LRC), k Nearest Neighbours ($k-$NN), Decision Tree Classifier (DTC), the Deep Neural Network (DNN), and Naive Bayes Classifier (NBC).



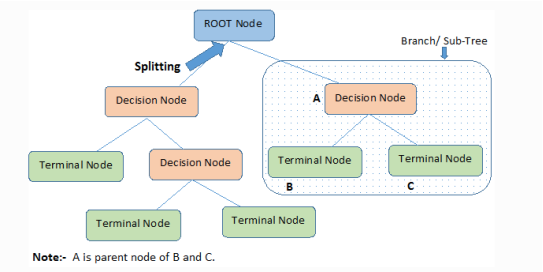
(a) SVM (linear) [31]



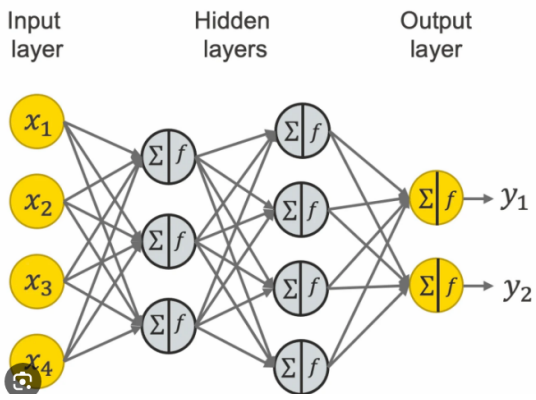
(b) Logistic Regression [32]



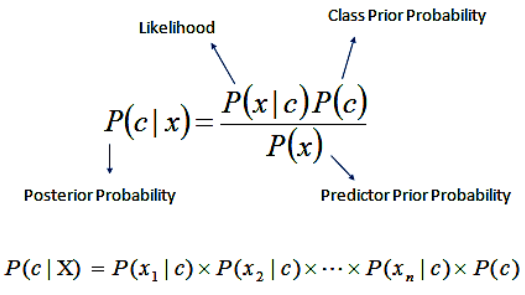
(c) $k-$NN [33]



(d) Decision Tree



(e) DNN [34]



(f) NBC [35]

Figure 4: Diagrams of various machine learning models.

These algorithms were used because they have been tested abundantly. The LRC is a baseline algorithm against which all the others may be compared.

Both the $k-$NN and DTC are modelled on forms of human reasoning. The $k-$NN is very similar to natural, intuitive human reasoning: looking at the item as a whole and categorising

new items based on its similarity to other categorised items. This is how humans make decisions. The DTC, however, attempts to simulate a more formal system of human reasoning. It models a type of filtering system by asking questions about a certain item to narrow down its classification.

The NBC represents data-driven algorithms. It's accuracy increases significantly with data due to the central limit theorem. The DNN is a popular algorithm for classification and has many use-cases. The SVM, when used alongside the DTC, provides good representation of non-parametric algorithms.

4.1 Initial Implementation Development and Model Hyperparameters

Each model was first implemented using a `baseModel` class along with six model classes: `LRC(baseModel)`, `SVM(baseModel)`, `NBC(baseModel)`, `kNN(baseModel)`, `DTC(baseModel)`, and `DNN(baseModel)` that inherit from `baseModel`. This relation can be seen in **Figure 5**.

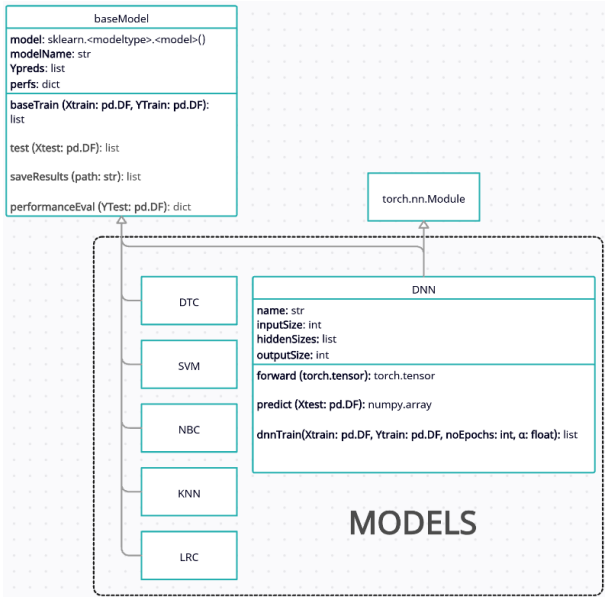


Figure 5: UML Class Diagram of the Models. Note: `torch.nn.Module` is the PyTorch DNN class, and `pd.DF` is the Panda's DataFrame class. **The full code can be found in Appendix 2.**

Next, a simple data preprocessing script was implemented so that the models could be trained and tested, alongside a script for computing and visualising performance metrics.

4.1.1 Hyperparameters

Table 2 shows the hyper parameters chosen for each model. [36] found that three sigmoid function-activated hidden layers with the RMS Propogation (RMSProp) provide a high accuracy model (on the WBCD). Further, $\alpha = 0.001$ is generally found to be the best learning rate with RMSProp [37].

Table 2: Key hyper-parameters used in this study

Model	**Hyper-parameter(s)**	**Value(s)**
NBC	Class priors	Uniform (equal weighting)
SVM-L	Kernel	Linear
	Regularisation C	1.0
k-NN	Search algorithm	Ball-Tree
	Distance metric	Manhattan (L_1)
	Number of neighbours k	*TBC – pick, e.g., 5*
DTC	Split criterion	Gini index
DNN [36]	Learning rate α	0.001
	Optimiser	RMSProp
	Epochs	3 000
	Hidden layers	3 fully-connected (sigmoid)
	Input nodes	6 (post-CFS)

5 Experiments and Methodology

The implementations and the experimental methodologies of the pre-processing and ML pipeline discussed in **Chapter 3** are described here.

5.1 Optimising Feature Selection

Algorithm 5 (CFS) (**Appendix A.1.1**) was implemented in Python using Pandas' in-built `DataFrame.corr()` method and `list.sort(ascending = False)` to obtain the relevancy of each feature, allowing the algorithm to discard redundant features. But, to find the most *optimal* number of selected features, the optimal number must first be defined:

Definition 5.1 (**Accuracy Ceiling**).
Let D be a dataset and $p \in \mathbb{N}_{>0}$ be the number of features of D. A model $Y_p = M(D_p)$ is trained on D_p a number of times where the value of p is varied. $\forall p$, the accuracy of the set of predictions Y_p is α_p. Then,

$$\exists\, p_O \in \mathbb{N}_{>0} \text{ s.th. } \forall\, p > p_O,\ \alpha_{p_O} \geq \alpha_p.$$

Any p that satisfies the above condition is called the most optimal p and is denoted as p_O. Further, α_{p_O} is called the accuracy ceiling of the model M.

Now, an intuitive experiment can be devised from which the *average* most optimal k can be found. The premise of this experiment is as follows:

1. Let p be 1 (as $0 < p \leq p_{\max}$, where $p_{\max}$ is the maximum number of features).
2. Run all models using the same p features (as selected by **Algorithm 1**).
3. Record the accuracies for each model.
4. The experiment can be stopped if $p = n$.

5. Increment p and repeat from step 2.

A chart is then plotted to show the general trend of accuracies with regard to features for each model. p_O is identified for each model (**Definition 5.1**). Finally, the average most optimal p-number of features for the whole larger experiment can be founded by finding the mean of all p_O's, rounded to the nearest integer.

5.2 Baseline Breast Tumour Classification with ML

This experiment is to capture the performance of each model on the WBCD Dataset using only FS, which is the minimal necessary preprocessing. By choosing $p = p_O = 6$ (see **Chapter 6.1**), the models can be trained on the most optimum number of features. FS is carried out only on the *training* dataset. Then, the same p_O features are kept in the testing set (without using feature selection).

5.3 Selecting Debiasing Techniques

The Mammography dataset has a 97%/3% split between benign/malignant classes (unlike the relatively balanced-WBCD). Hence, the four debiasing techniques were tested on the Mammography dataset. These techniques implemented using the `imblearn / imbalance-learn` module in Python. Then, a debiasing controller function was implemented and integrated into the rest of the preprocessing code.

The following method was used to collect debiasing results:

1. Train and test all the models on data set 2 with **no** debiasing.
2. Choose one technique out of: random over sampling, random under sampling, SMOTE, and ADASYN.
3. Train and test all models on data set 2 with the chosen debiasing technique.
4. Record and plot the performance metrics for all models.
5. Repeat until all techniques have been tested and results recorded.

5.4 Testing Domain Adaptation for Transfer Learning

The CORAL method (**Algorithm 3**) is implemented in Python using the Numpy library and their covariance function.

Methodology:

1. Train the model on the source domain dataset (WBCD).
2. Test the model on the "un-CORAL-ed" target domain dataset (Mammography dataset) and record the performance metrics for each model
3. Test the model on the "R-CORAL-ed" target domain dataset (Mammography dataset) and record the performance metrics for each model.

5.5 Performance Metrics

The performance metrics used in this project are accuracy, sensitivity, specificity, and precision (**Definition A.5**, **Appendix A.3**). Accuracy provides an indication of the general capability of the model. However in clinical contexts particularly sensitivity, and, to an extent specificity, are very important as they provide information on how well the model(s) can predict negative and positive classes correctly. Recall (mathematically identical to sensitivity) is also included, as the term 'sensitivity' is typically only used in medical contexts.

Sensitivity is, arguably, the most important metric in medical contexts. This is because a *high* sensitivity indicates that the number of false negative predictions are *low*. This is very important in medical contexts as a higher positive prediction rate - even if there are lots of false positive ones – is preferable, as it is much safer to predict that someone has a disease (enabling further investigation), than to miss it.

6 Results

6.1 Optimising Feature Selection - Results

The CFS algorithm was run and each model tested with various numbers of features ($p_{\max} = 30$).

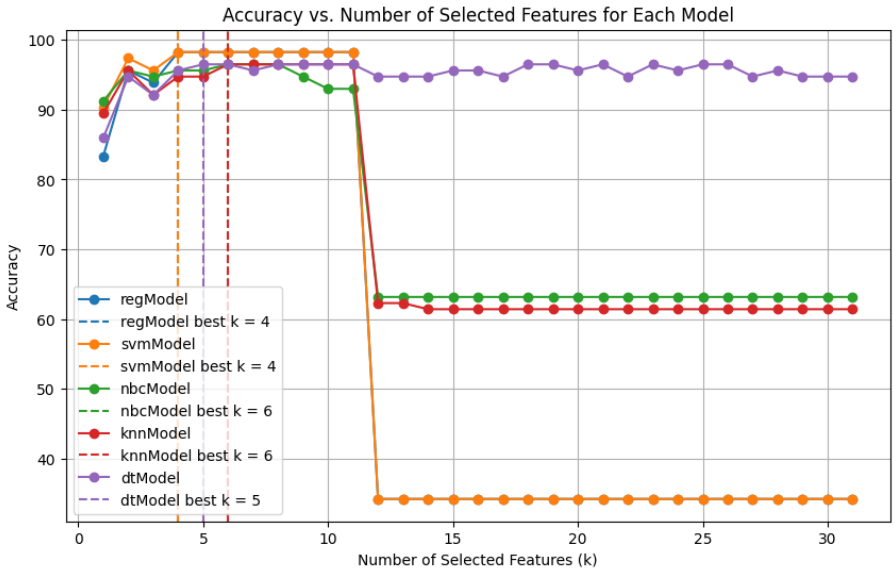


Figure 6: Number of features vs accuracies for all models (WBCD Dataset)

Figure 6 shows that each model has it's own α_{p_O}. The LRC and SVM models achieve their highest accuracy of 98.25% when $p = 4$. The DTC achieves it's when $p = 5$, and the NBC and $k-$NN when $p = 6$, with their highest accuracies falling just below at 96.49%.

Most models experience a significant drop of accuracy between $p = 10$ and $p = 13$. Even after this drop though, the accuracies of the SVM and LRC $\forall\ p > 6$ are very similar. The accuracy of both the NBC and $k-$NN also remain largely similar to each other for most values of p, even after their accuracy drop. However, the accuracy of the DTC model remains fairly consistent for all p after $p = 5$.

All models are at their maximum accuracy at $p = 6$, so this is the best number to use for the CFS algorithm going forward.

6.2 Baseline Classification With ML - Results

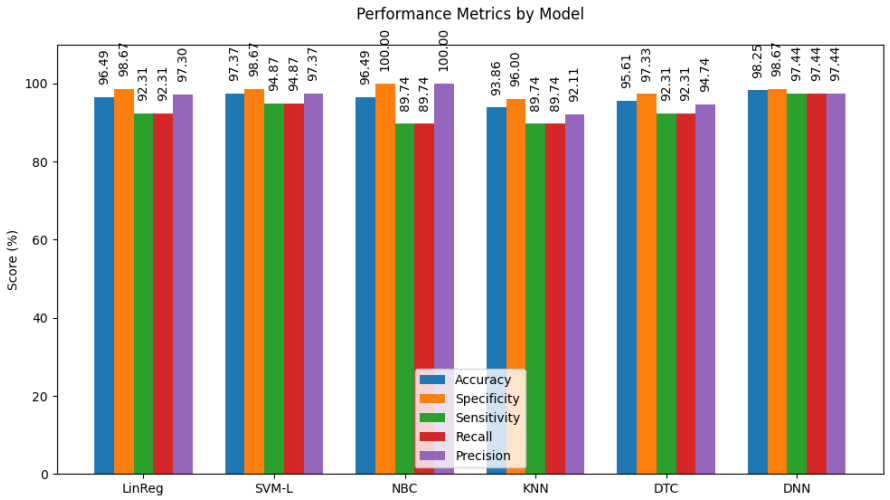


Figure 7: Performance of each model on the WBCD Dataset

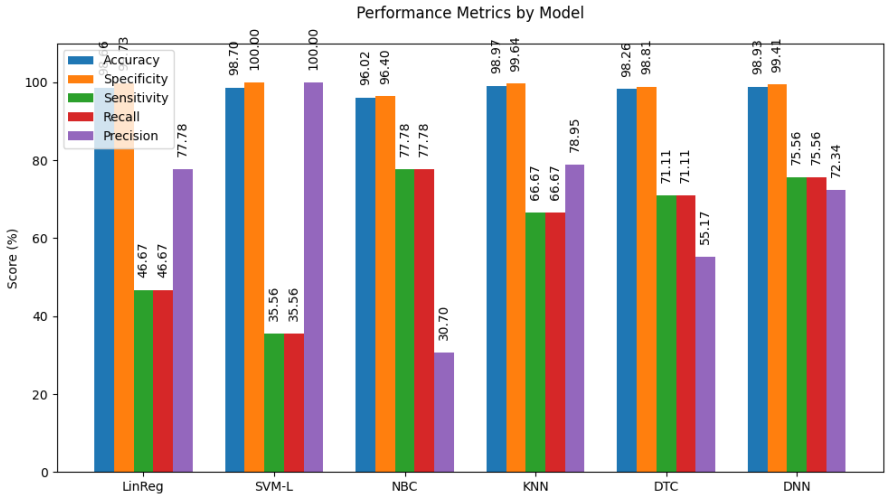


Figure 8: Performance of each model on the Mammography dataset (n.b. no debiasing involved)

For the WBCD, (**Figure 7**), all models perform to a relatively high standard across all performance metrics. Sensitivity, along with precision and accuracy, can provide the most insight into performance. Here, there is little variation between the models in terms of sensitivity, precision, and accuracy. All models are excellent in terms of their overall classification ability, as implied by the high accuracy. The DNN scores the highest accuracy at 98.25%, with the average across all models being 96.35%. The average sensitivity is also very strong, at 92.74%. This indicates that each model is unlikely to predict a false negative, and hence unlikely to miss a malignant tumour. Recall, although identical to sensitivity is included in the tables for completion (see **Chapter 5.5**). Precision indicates the number of malignant classifications that were actually correct. All models have strong precisions (with the NBC obtaining 100%), indicating that positive predictions are highly accurate. The LRC model also scores very highly across these important metrics and would thus make a good benchmark against which to compare the other models.

For the mammography dataset (**Figure 8**), accuracy and even specificity are very high across all models. However, the main issue is a sensitivity/recall (a key metric for medical applications, see **Chapter 5.5**) of just 62.23%. Sensitivity measures the proportion of actual malignant cases that were correctly classified. It can therefore be interpreted as a direct gauge of the models' 'safety' (false positives are 'safer' than false negatives). This low sensitivity demonstrates that biased data can be misleading with strong accuracies being due to data imbalance rather than the true-positive classification ability.

6.3 Selecting Debiasing Techniques (With Mammography Data) - Results

Table 3: Accuracy (%)

Sampling Tech.	LRC	SVM	NBC	KNN	DTC	DNN
SMOTE	90.17	90.08	81.63	95.89	97.68	95.84
ADASYN	83.10	81.05	74.47	95.08	97.41	93.25
Under	90.61	92.67	87.71	92.45	86.63	95.26
Over	89.54	90.39	83.33	97.90	98.30	95.49
No Debiasing	**98.66**	**98.70**	**96.02**	**98.97**	**98.26**	**75.56**

Table 4: Sensitivity (%)

Sampling Tech.	LRC	SVM	NBC	KNN	DTC	DNN
SMOTE	95.56	100.00	100.00	95.89	75.56	97.78
ADASYN	100.00	100.00	100.00	91.11	75.56	97.78
Under	93.33	93.33	100.00	95.56	93.33	97.78
Over	95.56	95.56	100.00	86.67	48.89	97.78
No Debiasing	**46.67**	**35.56**	**77.78**	**66.67**	**71.11**	**75.56**

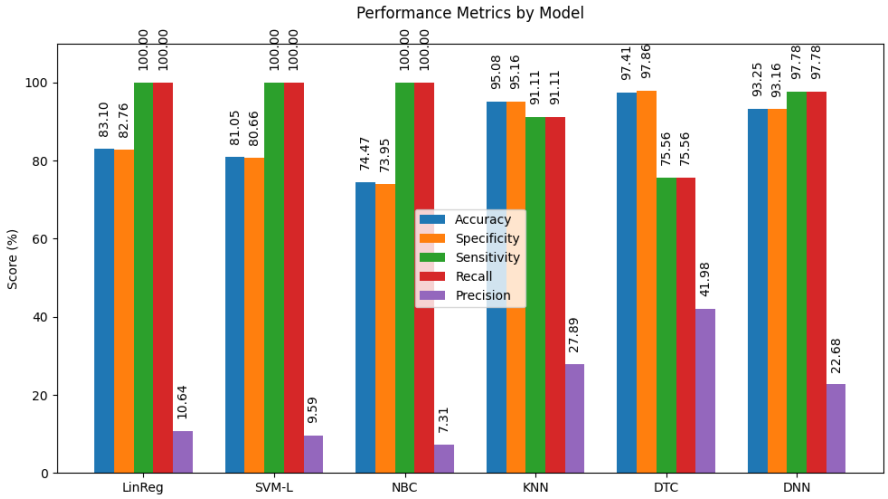
Key: Green - best, yellow - second best, red - worst (in terms of technique for each model).

Table 3 shows the accuracies for all models with each debiasing technique. All debiasing techniques lead to a significant drop in accuracy. ADASYN causes the largest drop for four of the models, with the drop in NBC being significantly more than in others, and the drop in DTC accuracy being only 0.67%. Random undersampling causes the least accuracy drop for three of the models. The only model to have an accuracy increase was the DTC with Random Oversampling (by 0.31%). Random Oversampling and SMOTE also caused a moderate accuracy drop. Between them, though, SMOTE has the larger accuracy drop for a model (NBC). This drop in accuracy does not necessarily invalidate the advantages of debiasing as a concept, as can be seen from the changes in sensitivity discussed below.

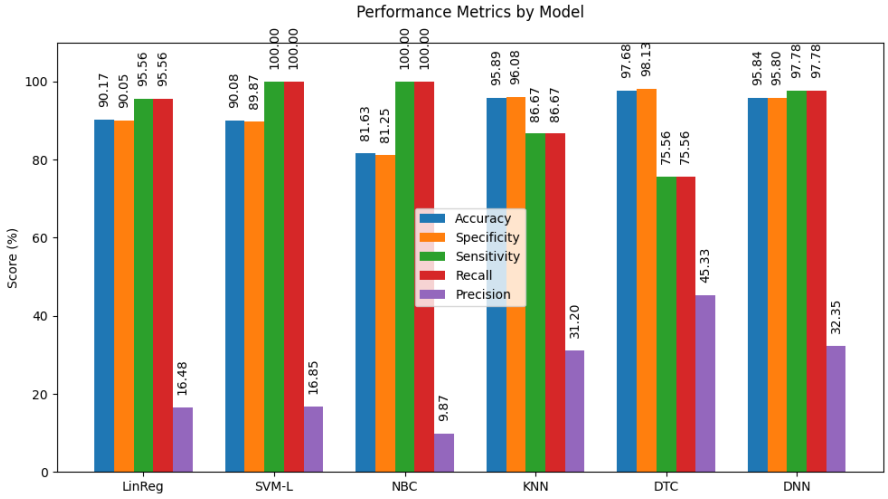
Table 4 shows a significant increase in sensitivity for all models (except the DTC) with any debiasing technique. ADASYN leads to an impressive increase in sensitivity (100% for three models and 97.78% for the DNN). SMOTE too causes an increase in sensitivity that is nearly as good. However, random oversampling causes a major drop in sensitivity for the DTC. In fact, the

techniques that yielded the lowest sensitivities were random under-sampling and over-sampling. Under-sampling yields a lower sensitivity for the linear-relationship based algorithms, but over-sampling is worse for 'human-reasoning' algorithms.

The accuracy drop can be attributed to the dataset balancing. Re-balancing teaches the models to expect a 50-50 class split, so on the still-skewed test set they predict malignant more often. Although sensitivity rises, specificity—and with it accuracy—falls. Because identifying malignant instances (reflected by sensitivity) is paramount, ADASYN offers the safest overall performance, with SMOTE close behind.



(a) Model performance with **ADASYN** debiasing on Mammography dataset.

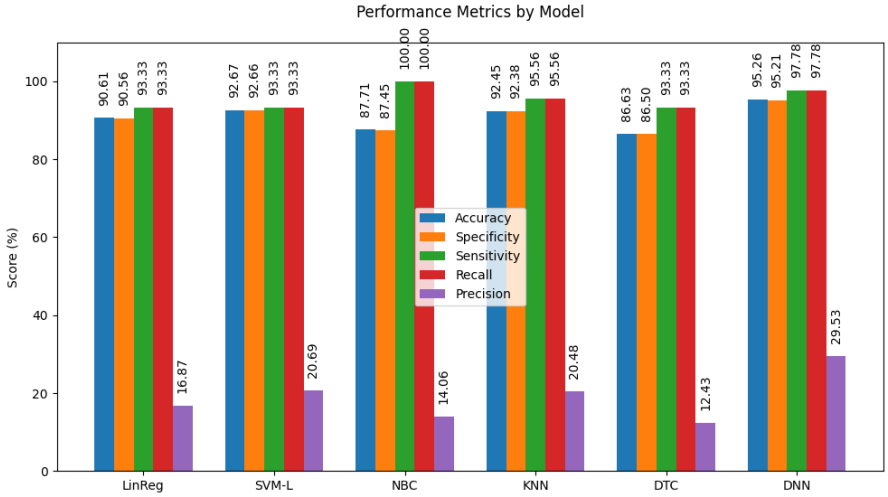


(b) Model performance with **SMOTE** debiasing on Mammography dataset.

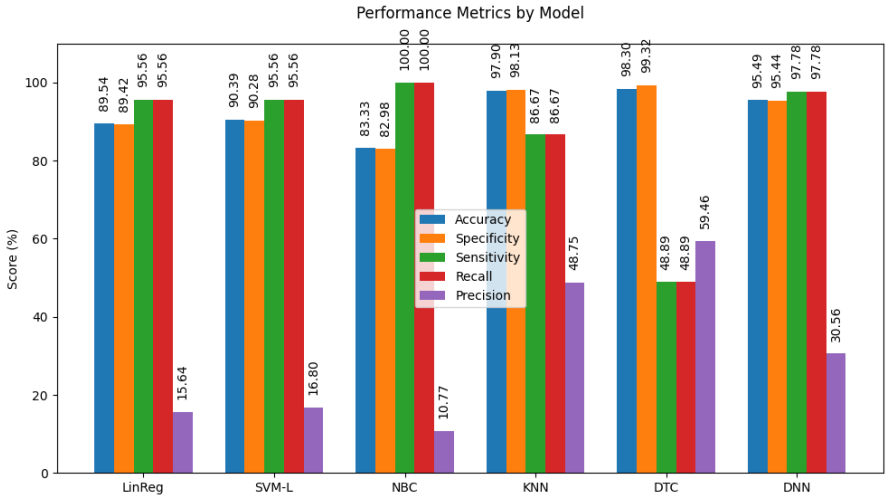
Figure 9: Performance of models on mammography dataset with synthesis-based debiasing.

Figures 9 and 10 show all models' metrics for each debiasing technique. Precision (reflecting accuracy of positive predictions) is low across all models with all debiasing techniques. Although far less important than sensitivity as a metric here, it is unfavourable to have very low precision and hence it is important to discuss this.

Specificity and accuracy for all models drops with debiasing. However, the drop is only small with specificity scores still being relatively high. Applying ADASYN results in the lowest specificity across three of the models (LRC (82.76%), SVM (80.66%), and NBC (73.95%)) with significant accuracy drops discussed previously. This indicates that the models are 'mass-classifying' instances as malignant leading to a high false positive (FP) rate. And, from **Definition A.5**, $\text{prec}(P) \propto \frac{1}{TP+FP}$. The models have thus become highly over-sensitive to the malignant class as a direct result of implementing debiasing techniques. The worst precision score was with NBC-ADASYN at 7.31%. However, the DTC, for all techniques, scores the highest precision, with the DTC-Oversampling combination scoring 59.46%.



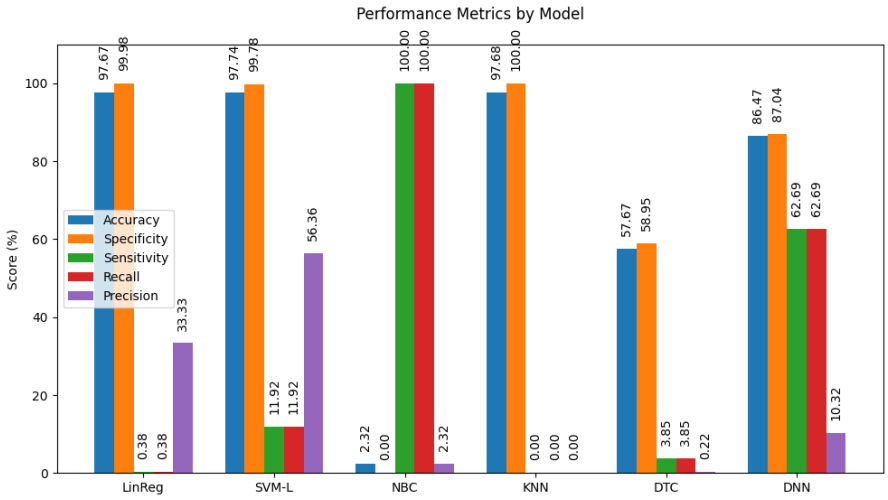
(a) Model performance with **Random Under Sampling** debiasing on Mammography dataset.



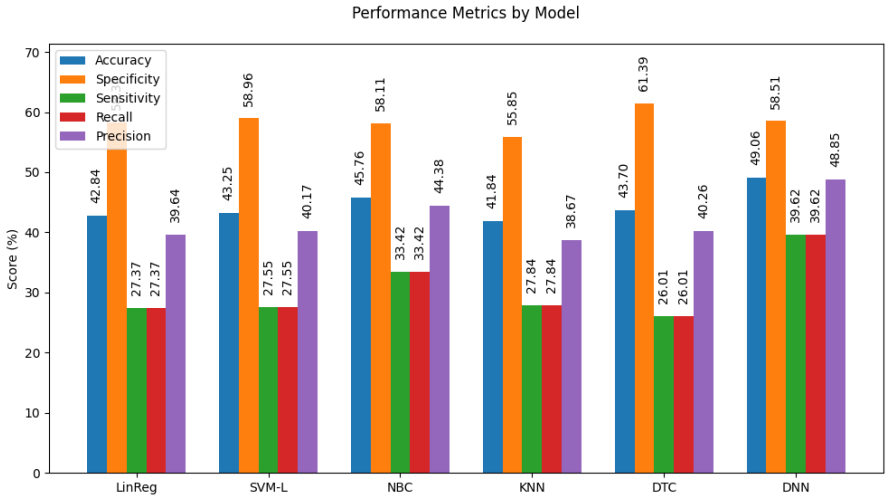
(b) Model performance with **Random Over Sampling** debiasing on Mammography dataset.

Figure 10: Performance of models on mammography dataset with random-based debiasing.

6.4 Testing Domain Adaptation - Results



(a) Model performance with **no domain adaptation**.



(b) Model performance with **R-CORAL**.

Figure 11: Performance of WBCD-trained models on mammography set.

Figure 11 shows the models' transfer learning ability *with* DA (**Figure 11b**) and *without* DA (**Figure 11a**). Without DA, most models scored extremely highly in specificity and accuracy but less than 5% in sensitivity; this indicates that they 'prefer' predicting false negatives than false positives - which is highly unsafe as malignancies may be missed. The NBC and DNN models are the only one able to score higher than 60% in sensitivity.

Figure 11b shows the models' performance after realigning the domains using R-CORAL. The models could now more effectively apply the patterns learnt in the WBCD dataset to the Mammography dataset. The specificity has taken a moderate hit here in comparison to 'without DA' (from 99% to 55% - 62%)., However the sensitivity has significantly increased (each model scoring between 26% and 35%). This trade-off between the two is okay as it indicates the

models are safer with DA. With DA, the precision has also increased significantly (with the largest increase being in NBC with 42.06%), implying that the model is not simply 'mass-classifying', but attempting to more accurately classify positive cases. Contrarily, however, the SVM's precision decreases by 16.19% after applying R-CORAL.

7 Analysis & Discussion of Results

The Optimising Feature Selection experiment was to determine the best number of selected features p_O that would maximise the performance of all models. The SVM-L and LRC accuracies are identical for most values of p, this might be because they have the same underlying foundation: using linear regression to output a prediction value. The DTC maintains a consistent accuracy for all values of p. But each model varies in implementation, so each p_O is not consistent across all models. The LRC and SVM have a p_O of 4, while the NBC and kNN have a p_O of 6. $p_O = 6$ was chosen as the average most optimal p, as this is the point at which all models are at their accuracy ceilings. **This value is the one used for later experiments.**

Using $p = 6$, the CFS algorithm was applied, and the baseline classification ability of each model on the WBCD and Mammography datasets were observed. All models perform very well across the three important performance metrics (sensitivity, accuracy and precision). The LRC, though, maintained very consistent scores across all metrics. It provides a very good 'minimum performance' that other models can be compared against and try to improve beyond. It is also a relatively simple algorithm to implement, is computationally efficient to train [38], and can be applied to many different applications. Therefore, it would serve as a good benchmark model against which the others can be tested.

Amongst the different algorithms for classifying the WBCD. the precision metric is extremely high for all models (with the NBC scoring 100%). Essentially, it answers the question 'when the model predicts malignant, how often is it correct?' This indicates that the models can recognise a malignant tumour's pattern if there is *no doubt in its decision*, but if it is unsure it will likely predict benign. This is because, in its experience, there is *always* a higher chance of a given instance being benign, not malignant. This therefore implies that the model (as expected from a biased set) has not been exposed to a wide enough range of malignant tumours. The NBC and $k-$NN though, have a lower sensitivity (perhaps the most important metric) and so cannot be considered as the first-choice options for the best classifier of this dataset. The best algorithms were the DNN and SVM followed by the LRC as these yielded the highest sensitivity, while still maintaining excellent precisions and hence are unlikely to miss malignancy.

However, with the Mammography dataset there is a heavy bias causing a substantial drop in sensitivity. The models with the highest sensitivities here are the NBC, DNN, and SVM-L. The linear-based models largely fail at sensitivity here, possibly due to their decision boundary missing key instances. The human-based algorithms perform so much better because they learn from the whole dataset at once, rather than per instance. Also, here, the SVM-L model obtained a 100% precision - indicating that while it predicted false negatives more often, when it did predict a malignant tumour, it was absolutely correct. While this level of precision is invaluable, the SVM-L still takes many 'unsafe risks' (low sensitivity leads to missed positive classes). Therefore, the NBC and DNN are the best models to use for this application, unless debiasing is applied.

The effect of various debiasing methods on performance metrics of models with the Mammography dataset was observed next. ADASYN achieves a very impressive increase in sensitivity across all models, with SMOTE following closely behind. However there is a very significant drop in accuracy which can largely be attributed to the re-balancing. The models learn to recognise malignant tumour patterns better but sometimes may classify some benign tumours

incorrectly (lowering specificity). This is because they make the incorrect 'assumption' that the test dataset (representative of real data distributions) has a similar distribution to that of their training dataset, i.e. that real tumours have an equal occurrence in the real world. But as the models are much more capable of identifying malignant tumours correctly, there is increased sensitivity but lower specificity and accuracy.

In medical contexts though, sensitivity is regarded as more important than accuracy. ADASYN and SMOTE maintain the most consistent sensitivity scores across models, followed by random undersampling. ADASYN obtains 100% with three models and SMOTE with two. The objective was to find the best debiasing technique that delivers consistent model performance, regardless of the algorithm used.

From this, it may be concluded that ADASYN provides more models with higher sensitivities than any other technique, even though it provides the worst accuracy/specificity drops. Thus, ADASYN is the most effective debiasing technique, especially when the LRC, SVM, and NBC are used. The precision for all techniques is very low, and this should be investigated further as it too many FP predictions can have adverse effects on patients, too.

Domain Adaptation was tested to see if it would be an effective way of aiding in transfer learning ability. With no DA, the classifiers generalized poorly: they focused on clear benign patterns, and whenever they encountered ambiguous instances, they simply defaulted to the benign label. All models, except the NBC, have incredibly high specificities and accuracies, but extremely low sensitivities. This is because each model has been trained on a slightly biased data meaning it is more likely to classify an unseen instance as benign. This produces very good results in terms of accuracy and specificity when the models are tested on a significantly biased dataset, because if they 'guess' an instance to be benign, they are likely to be correct.

The application of R-CORAL aids performance greatly, sensitivity for all models improves significantly. This results in much safer transfer learning for all models. Precision improves as well indicating that many of the malignant predictions are correct. The specificity and accuracy after DA is consistent across models. The DNN and NBC, however, obtain the highest sensitivities (33.42% and 39.62%) and the highest precisions, making them the best classifiers for transfer learning.

Although the DNN and NBC are by far the *best* classifiers for transfer learning, they are still by no means 'good'. Therefore, none of them should be used in their current state due to their lack of safety (sensitivities are still low). However, R-CORAL, and DA in general, should be considered as an integral part and a powerful tool of medical classification problems going forward.

8 Conclusion & Evaluation

8.1 Conclusion

This project examined whether debiasing and domain-adaptation improve breast-tumour classification. Performance was measured with accuracy, specificity, sensitivity and precision, with sensitivity ranked most important so as not to miss malignancy. On the WBCD data, DNN and SVM had a high capability for identifying malignancy, while NBC had the highest precision (100%), indicating that many of the malignant predicitons are correct. The heavily imbalanced Mammography set exposed a need for debiasing. Here, ADASYN and SMOTE identified malignancies excellently, at a cost to benign prediction. All techniques hurt precision, meaning that more true benign tumours are incorrectly classed as malignant. This means that these patients may face further tests, and stress – even when they don't have cancer. This is, unfortunately, a side-effect of prioritising sensitivity. Finally, R-CORAL as DA technique can improve some performance metrics of models but it is not at a satisfactory stage yet.

8.2 Future Work

Work should be done to identify and remedy the reasons for ADASYN classifying many benign tumour as malignant even when given a balanced dataset). Other debiasing and domain adaptation techniques should be investigated with these datasets to provide a more well-rounded analysis.

Finally, more specific TL techniques may be implemented for deep learning models. More specifically, the use of fine tuning. This is done by first training the full DNN on the source dataset. Then, a new output layer (untrained) is attached to the base model (replacing the old output layer). The weights for the base layers are effectively 'frozen' (they don't change) and the model is trained on the target dataset. Finally, using a much smaller learning rate, the base layers are unfrozen and the whole model then trains on the mammography dataset.

For all techniques across all models, the precision metric dropped even more than with no debiasing. This is because the models may have 'mass-classified' instances as malignant leading to a high FP rate. And, from **Definition A.5**, $\text{prec}(P) \propto \frac{1}{TP+FP}$. This needs further investigating.

8.3 Project Evaluation

This project successfully implemented feature selection, debiasing, and domain adaptation, producing results that confirm both the promise and the current limits of ML-based breast tumour classification. Domain adaptation, though not yet delivering excellent performance, provides a solid baseline for future refinements.

Automated classification remains fallible, meaning without clinician oversight it can misclassify tumours. This risks false negative or false positive predictions that expose patients to needless anxiety, tests, and costs. Therefore, the models developed here should be viewed as decision-support tools: they flag suspicious cases that doctors then confirm, rather than replacing expert judgement.

During the project, unexpected outcomes were encountered (such as less than satisfactory performance). During certain periods, the project plan had to be changed in order to account for this lost time in researching risk mitigation methods. At one point it was found that the models were actually being tested on debiased data. This is a major risk issue as it does not necessarily prove that the models will perform well in the real world, as the test was tainted with artificial data. Therefore, the preprocessing pipeline was revised to only debias the training data and research was carried out to validate this method.

8.3.1 Personal Development & Self-Reflection

I built the models with established libraries— Sci Kit-Learn (for the classical algorithms), and PyTorch for the DNN—and iteratively developed the preprocessing pipeline to ensure each component worked as intended.

The project deepened my understanding of the underlying mathematics and sharpened my Python skills via the use of Numpy and Pandas. It allowed me to learn more about the PyTorch module, which I had never used before.

An error during the project was the codebases structure. I began by adding new modules to an ad-hoc script; midway through the project, the code became difficult to debug and unreadable. I rebuilt the entire codebase with an object-oriented design that enforces clear data flow and includes documentation for each class.

This experience taught me that meticulous up-front planning—class diagrams, detailed flow charts, and a testing strategy—is as important as the technical work itself.

References

[1] Cancer Research UK. “what is cancer?”. Cancer Research UK. Accessed 2025-05-20. [Online]. Available: https://www.cancerresearchuk.org/about-cancer/what-is-cancer

[2] ——. “cancer statistics for the uk”. Cancer Research UK. Accessed 2025-05-20. [Online]. Available: https://www.cancerresearchuk.org/health-professional/cancer-statistics-for-the-uk

[3] M. Aisha Patel, “Benign vs malignant tumors,” https://jamanetwork.com/journals/jamaoncology/fullarticle/2768634, 2020, accessed: 25 October 2024.

[4] G. e. a. Sharma, “Various types and management of breast cancer: An overview,” *Journal of Advanced Pharmaceutical Technology & Research*, vol. 1, no. 2, p. 109, 2010.

[5] “Breast cancer (female) symptoms and treatments,” https://www.nhsinform.scot/illnesses-and-conditions/cancer/cancer-types-in-adults/breast-cancer-female/, 2024, accessed: 25 October 2024.

[6] OWise Team. “understanding your breast cancer diagnosis: A guide”. OWise UK. Accessed 2025-04-30. [Online]. Available: https://owise.uk/understanding-breast-cancer-diagnosis/

[7] Cancer Research UK. “biopsy (tests and scans)”. Cancer Research UK. Accessed 2025-05-20. [Online]. Available: https://www.cancerresearchuk.org/about-cancer/tests-and-scans/biopsy

[8] L. Jiang, Z. Wu, X. Xu, Y. Zhan, X. Jin, L. Wang, and Y. Qiu, “Opportunities and challenges of artificial intelligence in the medical field: current application, emerging problems, and problem-solving strategies,” *Journal of International Medical Research*, vol. 49, no. 3, p. 03000605211000157, 2021, pMID: 33771068. [Online]. Available: https://doi.org/10.1177/03000605211000157

[9] Z. Feng and X. Hua, “Applications and current status of ai in the medical field,” *Journal of Physics: Conference Series*, vol. 2289, no. 1, p. 012030, jun 2022. [Online]. Available: https://dx.doi.org/10.1088/1742-6596/2289/1/012030

[10] D. A. Omondiagbe *et al.*, “Machine learning classification techniques for breast cancer diagnosis,” in *IOP Conference Series: Materials Science and Engineering*, vol. 495, 2019, p. 012033.

[11] M. Sewak, P. Vaidya, C.-C. Chan, and Z.-H. Duan, “Svm approach to breast cancer classification,” in *Proceedings of an IEEE Conference*, Aug 2007, accessed from https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=4392577.

[12] S. AhmedMedjahed, T. A. Saadi, and A. Benyettou, “Breast cancer diagnosis by using k-nearest neighbor with different distances and classification rules,” *International Journal of Computer Applications*, vol. 62, no. 1, pp. 1–5, Jan 2013.

[13] O. I. Obaid, M. A. Mohammed, M. K. A. Ghani, S. A. Mostafa, and F. T. AL-Dhief, “Evaluating the performance of machine learning techniques in the classification of wisconsin breast cancer,” *International Journal of Engineering & Technology*, vol. 7, no. 4.36, pp. 160–166, Dec 2018.

[14] L. Vig, "Comparative analysis of different classifiers for the wisconsin breast cancer dataset," *OALib*, vol. 1, no. 6, pp. 1–7, 2014.

[15] M. Amrane, S. Oukid, I. Gagaoua, and T. Ensari, "Breast cancer classification using machine learning," in *2018 Electric Electronics, Computer Science, Biomedical Engineerings' Meeting (EBBT)*, Istanbul, Turkey, 2018, pp. 1–4.

[16] A. M. Abdel-Zaher and A. M. Eldeib, "Breast cancer classification using deep belief networks," *Expert Systems with Applications*, vol. 46, pp. 139–144, Mar 2016.

[17] S. Khan, N. Islam, Z. Jan, I. Ud Din, and J. J. P. C. Rodrigues, "A novel deep learning based framework for the detection and classification of breast cancer using transfer learning," *Pattern Recognition Letters*, vol. 125, pp. 1–6, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167865519301059

[18] A. Seemendra, R. Singh, and S. Singh, "Breast cancer classification using transfer learning," 11 2020.

[19] UCI ML Repository, "Breast cancer wisconsin (diagnostic) data set," https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data, 1995, kaggle dataset, accessed February 25, 2025.

[20] K. S. Woods, J. L. Solka, C. E. Priebe, W. P. Kegelmeyer, C. C. Doss, and K. W. Bowyer, *Comparative Evaluation of Pattern Recognition Techniques for Detection of Microcalcifications in Mammography*, pp. 213–231. [Online]. Available: https://www.worldscientific.com/doi/abs/10.1142/9789812797834_0011

[21] P. Mishra, A. Biancolillo, J. M. Roger, F. Marini, and D. N. Rutledge, "New data preprocessing trends based on ensemble of multiple preprocessing techniques," *Trends in Analytical Chemistry*, vol. 132, p. 116045, 2020.

[22] s. developers, "KNNImputer — scikit-learn 1.6.1 documentation," https://scikit-learn.org/stable/modules/generated/sklearn.impute.KNNImputer.html, 2025, accessed: 2025-04-26.

[23] M. A. Hall, "Correlation-based feature selection for machine learning," Ph.D. dissertation, The University of Waikato, Department of Computer Science, Hamilton, New Zealand, 1999, ph.D. Thesis.

[24] H. He, Y. Bai, E. A. Garcia, and S. Li, "Adasyn: Adaptive synthetic sampling approach for imbalanced learning," in *2008 IEEE international joint conference on neural networks (IEEE world congress on computational intelligence)*. Ieee, 2008, pp. 1322–1328.

[25] R. Mohammed, J. Rawashdeh, and M. Abdullah, "Machine learning with oversampling and undersampling techniques: overview study and experimental results," pp. 243–248, 2020.

[26] Mohammed, Roweida and Rawashdeh, Jumanah and Abdullah, Malak, "Machine learning with oversampling and undersampling techniques: Overview study and experimental results," in *2020 11th International Conference on Information and Communication Systems (ICICS)*, 2020, pp. 243–248.

[27] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: synthetic minority over-sampling technique," *Journal of artificial intelligence research*, vol. 16, pp. 321–357, 2002.

[28] F. Zhuang, Z. Qi, K. Duan, D. Xi, Y. Zhu, H. Zhu, H. Xiong, and Q. He, "A comprehensive survey on transfer learning," *Proceedings of the IEEE*, vol. 109, no. 1, pp. 43–76, 2021.

[29] Adapt Python Team, "Adapt: Feature-based," https://adapt-python.github.io/adapt/contents.html#adapt-feature-based, accessed: 21 March 2025.

[30] B. Sun, J. Feng, and K. Saenko, "Correlation alignment for unsupervised domain adaptation," *Domain adaptation in computer vision applications*, pp. 153–171, 2017.

[31] Hesaraki, Saba. "support vector machines (svm) classifier". Medium.com. Accessed 2025-04-30. [Online]. Available: https://medium.com/@saba99/support-vector-machines-svm-classifier-f5413109bd10

[32] Sondi, Jackson Jacob. "logistic regression in r". Scaler.com. Accessed 2025-04-30. [Online]. Available: https://www.scaler.com/topics/r-logistic-regression/

[33] Shah, Rajvi. "introduction to k-nearest neighbors (knn) algorithm". Plainenglish.io. Accessed 2025-04-30. [Online]. Available: https://ai.plainenglish.io/introduction-to-k-nearest-neighbors-knn-algorithm-e8617a448fa8

[34] Melcher, Kathrin. "a friendly introduction to [deep] neural networks". KNIME.com. Accessed 2025-04-30. [Online]. Available: https://www.knime.com/blog/a-friendly-introduction-to-deep-neural-networks

[35] M. Mitzenmacher and E. Upfal, *Probability and Computing*, 2nd ed. Cambridge University Press, 2017 (2nd edition 2022), ch. 1, pp. 12 – 14.

[36] N. Darapureddy, N. Karatapu, and T. K. Battula, "Implementation of optimization algorithms on wisconsin breast cancer dataset using deep neural network," in *2019 4th International Conference on Recent Trends on Electronics, Information, Communication Technology (RTEICT)*, 2019, pp. 351–355.

[37] D. Irfan, T. S. Gunawan, and W. Wanayumini, "Comparison of sgd, rmsprop, and adam optimation in animal classification using cnns," *International Conference on Information Science and Technology Innovation (ICoSTEC)*, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:260646795

[38] J. C. Stoltzfus, "Logistic regression: A brief primer," *Academic Emergency Medicine*, vol. 18, no. 10, pp. 1099–1104. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1553-2712.2011.01185.x

[39] F. Author, S. Author, and T. Author, "What are sensitivity and specificity?" *Evidence-Based Nursing*, vol. 23, no. 1, pp. 2–?, YYYY. [Online]. Available: https://ebn.bmj.com/content/23/1/2

A Appendix A: Algorithms, Definitions, and Maths

Below is the mathematical set-wise definition for a dataset that will be used throughout the report.

Definition A.1 (Dataset).
Let the set $D = \{y_i, x_{i0}, ..., x_{ij}, ..., x_{ip}\}_{i=0}^{n}$ be a dataset of n instances, and $C = \{c_0, c_1, ..., c_k, ..., c_{m-1}\}$ be the set of m possible classes.

1. Each row in D is called an instance and is defined as

$$D_i = \{y_i, x_{i,0}, x_{i,1}, ...x_{i,j}, ..., x_{i,p}\}, 0 \leq i \in \mathbb{N}_{\geq \nu} \leq n - 1$$

2. $\forall\ D_i\ \exists$ a set X_i s.th. $X_i = D_i \setminus \{y_i\}$. That is, X_i is the set of all the feature observations for the i-th instance.

3. Let V_j be an n-length column vector. Each $v_{i,j} \in Vj$ is the i-th instance's observed value for the j-th feature. Therefore, each column V_j is called a feature of D.

4. C is the set of all *possible* classes, while Y is the set or column vector of all dianogses for each D_i. It therefore follows that $Y \subset C$.

5. $Y' = \{y'_0, y'_1, ..., y'_i, ...y'_n\} \subseteq C$ is the set of all predictions made by some ML model on some $D-$like instance, X.

A.1 Correlation Feature Selection

Here, the mathematical definitions for *relevance*, *redundance*, and *a useful feature* are defined. Then, the CFS algorithm (**Algorithm 5**) is defined. See **Chapter 3.2**.

Definition A.2 (**Relevance** - [23])**.**
Let the set D be a dataset. A feature V_j of D is said to be relevant if $\exists$ some observation $v_{i,j} \in V_j$ and classification $y_i \in Y$ (where $v_{i,j}, y_i \in D_i$) s.th.

$$P(y_i' = y_i | V = v) \neq P(y_i' = y_i)$$

Alternatively, a feature is relevant if the correlation between it and the predictive class, $r_{V_j,Y}$, is s.th. $r_{V_j,Y} > 0$.

Definition A.3 (**Redundancy** - [23])**.**
Given a data set D (by **Definition 3.1**), any feature V_j of D is known as 'redundant' if there is a correlation $r_{V_j,W} \neq 0$ between V_j and another feature W of D.

Definition A.4 (**Useful Features**)**.**
Given a dataset D, the predictive class Y of D (by **Definition 3.1**), and two threshold values $0 \leq \tau_{\text{red}}, \tau_{\text{rel}} \in \mathbb{R} \leq 1$, any feature V of D is *useful* if

1. The correlation $r_{V,Y}$ is s.th. $r_{V,Y} > \tau_{\text{rel}}$
   $\implies$ The feature *is relevant*.

2. $\forall r_{V,W} \in R_V, r_{V,W} < \tau_{\text{red}}$.
   Where $R_V = \{r_{V,W} | \forall \text{ feature } W \in D \text{ s.th. } W \neq V\}$
   $\implies$ The feature is *not redundant*.

The values of τ_{red} and τ_{rel} can be found using trial and error.

A.1.1 CFS Algorithm

The CFS algorithm uses the **PopulateR** algorithm (**Algorithm 4**) to populate a correlation matrix.

Algorithm 4 PopulateR

Require: A dataset matrix $D = [y_i, x_{i,o}, x_{i,1}, ..., x_{i,j}, ..., x_{i,p}]_{i=0}^{n}$ of n instances and $p+1$ features, and a number of selected features, $k \in \mathbb{N}$

Ensure: A $p + 2$ by $p + 1$ matrix R holding the correlation of every feature with every other feature.

1:

2: *Let R be a $p + 2$ by $p + 1$ matrix holding the correlation of a feature V_i with each other feature V_j* (as defined by **Definition A.1.3**).

3:

$$R \leftarrow \begin{bmatrix} r_{0,y} & r_{0,0} & r_{0,1} & \cdots & r_{0,p+1} \\ r_{1,y} & r_{1,0} & r_{1,1} & \cdots & r_{1,p+1} \\ r_{2,y} & r_{2,0} & r_{2,1} & \cdots & r_{2,p+1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ r_{p,y} & r_{p,0} & r_{p,1} & \cdots & r_{p,p+1} \end{bmatrix}$$

Now calculate the $r_{i,j}$ $\forall$ (V_i, V_j) pairs in D

4: **for** $i \in \mathbb{N}_0$ **from** $i = 0$ **to** $i = p$ **do**
5: $r_{V,Y} \leftarrow \mathbf{PCC}(V_i, Y)$
6: $R_{i,0} \leftarrow r_{V,Y}$
7: **end for**
8:
9: **for** $i \in \mathbb{N}_0$ **from** $i = 0$ **to** $i = p$ **do**
10: **for** $j \in \mathbb{N}_{\geq i}$ **from** $j = 0$ **to** $j = p$ **do**
11: **if** R already has an entry in (j, i) **then**
12: $R_{i,j} \leftarrow R_{j,i}$
13: **Continue**
14: **end if**
15: *Let $r_{i,j}$ be the **PCC** between features V_i and V_j.*
16: $r_{i,j} \leftarrow \mathbf{PCC}(V_i, V_j)$
17: $R_{i,j} \leftarrow r_{i,j}$
18: **end for**
19: **end for**
20: **return** R

Algorithm 5 CFS

Require: A dataset matrix $D = [y_i, x_{i,o}, x_{i,1}, ..., x_{i,j}, ..., x_{i,p}]_{i=0}^{n}$ of n instances and $p+1$ features, and a number of selected features, $k \in \mathbb{N}$.

Ensure: A column vector F_{selected} of selected features s.th. $|F_{\text{selected}}| = p + 1$

$R \leftarrow$ **PopulateR**(D) ***Algorithm 4***

Let τ_{rel} and τ_{red} be as defined in ***Definition A.4***

Let $F_{selected}$ be an empty row vector of the indices of the final selected features. $|F_{selected}| = p + 1$.

$F_{\text{selected}} \leftarrow \begin{bmatrix} 0 & 0 & 0 & \cdots \end{bmatrix}$

for $i \in \mathbb{N}_0$ **from** $i = 0$ **to** $i = p$ **do**
    $r_{V,Y} \leftarrow R_{i,0}$
    if $r_{V,Y} \geq \tau_{\text{rel}}$ **then**
        $F_{\text{selected}}[i] \leftarrow 1$
    end if
end for

for $i \in \mathbb{N}_0$ **from** $i = 0$ **to** $i = p$ **do**
    if $F_{\text{selected}}[i] = 1$ **then**
        Let $T_{V_i} \in R$ be the row vector of correlations of V_i with all other features in D.
        for $j \in \mathbb{N}_0$ **from** $j = 0$ **to** $j = p$ **do**
            Let k_F be the number of 1's in $F_{selected}$.
            if $k_F \leq k$ **then**
                return F_{selected}
            else if $F_{\text{selected}}[j] = 1 \ \wedge \ T_{V_i}[j] \geq \tau_{\text{red}}$ **then**
                $F_{\text{selected}}[j] \leftarrow 0$
            end if
        end for
    end if
end for

return F_{selected}

A.2 ADASYN Debiasing Algorithm

Algorithm 6 ADASYN - [24]

Require: A set of minority-classified instances T s.th. $T \subseteq D$ (as in **Definition 3.1**), a number $k \in \mathbb{N}$ of nearest neighbours, and a threshold $0 < d_{\text{th}} \leq 1$ for the degree of imbalance.

Ensure: A set T_{syn} of synthetic samples to be added to D. $|T_{\text{syn}}| = \frac{N}{100} \cdot |T|$

Let m_s, m_l be the number of minority and majority class-instances, respectively.
$m_s \leftarrow |T|$
$m_l \leftarrow |T \setminus D|$
Let d be the degree of class imbalance s.th. $d \in (0, 1]$
$d \leftarrow \frac{m_s}{m_l}$

if $d < d_{\text{th}}$ **then**
    Let G be the number of minority-class instances to be generated over the whole dataset.
    Let $\beta \in [0, 1]$ be the desired balance level. $\beta = 1 \implies$ a fully balanced set.
    $G \leftarrow (m_s - m_l) \cdot \beta$

    Let $\mathbf{r}$ be a vector of length m_s.
    for $i \in \mathbb{N}_0$ **from** $i = 0$ **to** $i = m_s - 1$ **do**
        Let T_i be some instance in T.
        *Let **KNN**(k, T, T_i) denote the function for a k-Nearest Neighbours algorithm.*
        Let neighbours$_i$ be the k-length row vector of the indices(in T) of the k-nearest neighbours to T_i s.th. $T_n \subseteq T$, where $n \in$ neighbours$_i$.
        neighbours$_i \leftarrow$ **KNN**(k, T, T_i)

        Let χ_i be the number of neighbours to T_i that are in $T \setminus D$.
        $r_i \leftarrow \frac{|\chi_i|}{k} \in [0, 1]$
    end for

    Let $\hat{\mathbf{r}}$ be the vector holding all normalised r_i's
    $\hat{\mathbf{r}} \leftarrow$ [**normalise**$(\mathbf{r}, r_0)$, **normalise**$(\mathbf{r}, r_1)$, ...]
    Let $\mathbf{g}$ ($|\mathbf{g}| = m_s$) be the vector of the number of synthetic instances for each T_i.
    $\mathbf{g} \leftarrow [g_0, g_1, ...]$ *where* $g_i = \hat{r}_i \cdot G$
    $T_{\text{syn}} \leftarrow$ **SMOTE**$(T, \mathbf{g}, k_{\text{SMOTE}})$
end if
return T_{syn}

A.3 Other Definitions

Each definition below uses $P = \{TP, TN, FP, FN\}$, the set of prediction measurements. Each element of P is type of prediction a given model makes. They mean the number of **t**rue **p**ositive, **t**rue **n**egative, **f**alse positive, and **f**alse negative predictions, respectively. Therefore, $|P| = |Y'|$.

Definition A.5 (**Performance Metrics** - [39])**.**
Given some prediction measurements $P = \{TP, TN, FP, FN\}$, the accuracy acc(P),

precision prec(P), sensitivity sens(P), and specificity spec(P) are defined as follows:

$$\text{acc}(P) = \frac{TP + TN}{TN + TN + FP + FN}$$

$$\text{prec}(P) = \frac{TP}{TP + FP}$$

$$\text{sens}(P) = \frac{TP}{TP + FN}$$

$$\text{spec}(P) = \frac{TN}{TN + FN}$$

Remark. **Sensitivity** describes the model's ability to correctly predict a *malignant* tumour. On the other hand, **Specificity** describes the model's ability to correctly predict a *benign* tumour [39]. High sensitivity indicates that the number of false negative predictions are *low*. A high specificity indicates that the number of false positive predictions are *low*.

B Appendix B: Code Implementation

B.1 Information

Language: Python Libraries: Sci-Kit Learn, PyTorch, Matplotlib, Pandas, Numpy, Seaborn, Imblearn.

B.2 Model Clases

Based on the class diagram in **Figure 5**. **The full code can be found at the GitHub repository**. The following is *only* `models.py` from the repository.

```
# 11/04/2025
# V2
# ================================== IMPORTS
    ================================== #
from imports import *
import torch
import torch.nn as nn
import torch.optim as optim
#
    ================================================================================
    #

# ================================== BASE MODEL
    ================================== #
class baseModel:
  def __init__(self, model, modelName):
    self.model = model
    self.modelName = modelName
    self.Ypreds = []
    self.perfs = {
      "stats": { "TP": 0, "TN": 0, "FP": 0, "FN": 0 },
```

```
    "perfmetrics": {
      "accuracy": 0.0,
      "specificity": 0.0, # how many negative cases are correctly classified
      "sensitivity": 0.0, # how many positive cases are correctly classified
      "recall": 0.0, # how many actual cases are correctly classified
      "precision": 0.0 # how many correct predictions out of all predictions.
    }
  }

def basetrain(self, Xtrain, Ytrain):
  if self.modelName.lower() in ["dnn", "neuralnet", "ann"]:
    return self.dnnTrain(Xtrain, Ytrain, noEpochs = 3000,  = 0.001) # for
        alpha see COMPARISON OF SGD, RMSProp, AND ADAM OPTIMATION IN ANIMAL
        CLASSIFICATION USING CNNsDesi Irfan1, Teddy Surya Gunawan2,
        Wanayumini3
  else:
    try:
      self.model.fit(Xtrain, Ytrain)
      print(self.model)
      #if self.modelName == "dtModel":
      #  plt.figure(figsize=(20, 10), dpi=200) # bigger figure and higher DPI
      #  skl.tree.plot_tree(self.model, filled=True)
      #  plt.show()
      return [True, None, self]
    except Exception as e:
      return [False, e, self]

def test(self, Xtest):
  try:
    if self.modelName.lower() in ["dnn", "neuralnet", "ann"]:
      self.Ypreds = self.predict(Xtest)
    else: self.Ypreds = self.model.predict(Xtest)
    return [True, None, self.Ypreds]
  except Exception as e:
    return [False, e, None]

def saveResults(self, path):
  try:
    data = {
      "modelName": self.modelName,
      "performance": self.perfs,
      "Ypreds": self.Ypreds.tolist()
    }

    with open(path, "w+") as savefile:
      json.dump(data, savefile)
    return [True, None, data]
  except Exception as e:
    return [False, e, None]

def performanceEval(self, Ytest) -> dict:
```

```
    # check if model has been tested
    if len(self.Ypreds) != Ytest.shape[0]: raise Exception("Please test the
        model before trying to evaluate performance.")

    perfmetrics = {}
    # accuracy
    perfmetrics['accuracy'] = skl.metrics.accuracy_score(Ytest, self.Ypreds) *
        100
    print(self.modelName, " accuracy (in %):", perfmetrics['accuracy'])

    # confusion matrix
    print(pd.crosstab(Ytest, self.Ypreds, rownames = ["Actual"], colnames =
        ["Predicted"], margins = True));

    # Find TP, FP, TN, FN for specificity and sensitivity
    # We take 1 (M) as positive and 0 (B) as negative.

    Ytest = np.array(Ytest.values)
    vals = { 'TP': 0, 'FP': 0, 'TN': 0, 'FN': 0 }
    for i in range(len(self.Ypreds)):
      if(self.Ypreds[i] == 1 and Ytest[i] == 1): #TP
        vals['TP'] += 1
      elif(self.Ypreds[i] == 0 and Ytest[i] == 0): #TN
        vals['TN'] += 1
      elif(self.Ypreds[i] == 1 and Ytest[i] == 0): #FP
        vals['FP'] += 1
      elif(self.Ypreds[i] == 0 and Ytest[i] == 1): #FN
        vals['FN'] += 1

    # if statements to avoid division by zero.
    perfmetrics['sensitivity'] = 0 if (vals['TP'] + vals['FN']) == 0 else
        (vals['TP']/(vals['TP']+vals['FN']))*100
    perfmetrics['specificity'] = 0 if (vals['TN'] + vals['FP']) == 0 else
        (vals['TN']/(vals['TN']+vals['FP']))*100
    perfmetrics['recall'] = perfmetrics['sensitivity']
    perfmetrics['precision'] = 0 if (vals['TP'] + vals['FP']) == 0 else
        (vals['TP']/(vals['TP']+vals['FP']))*100

    self.perfs["stats"] = vals
    self.perfs["perfmetrics"] = perfmetrics
    return perfmetrics

#
    ================================================================================
    #
# ================================== MODEL CLASSES
    ================================== #

class LRC(baseModel):
  def __init__(self, name = "regModel"):
    super().__init__(skl.linear_model.LogisticRegression(random_state=0), name)
```

```
class SVM(baseModel):
  def __init__(self, name = "svmModel"):
    super().__init__(skl.svm.SVC(kernel = "linear"), name)


class NBC(baseModel):
  def __init__(self, name = "nbcModel"):
    super().__init__(skl.naive_bayes.GaussianNB(), name)


class KNN(baseModel):
  def __init__(self, name = "knnModel"):
    super().__init__(skl.neighbors.KNeighborsClassifier(algorithm =
        'ball_tree', metric = 'manhattan'), name)

# SKLearn defaultly uses an optimised implementation of the CART decision
    tree algorithm.
class DTC(baseModel):
  def __init__(self, name = "dtcModel"):
    super().__init__(skl.tree.DecisionTreeClassifier(criterion='gini',
        splitter='best'), name)

class DNN(nn.Module, baseModel):
  def __init__(self, inputSize, hiddenSizes, outputSize, name = "DNN"):
    self.inputSize = inputSize
    self.hiddenSizes = hiddenSizes
    self.outputSize = outputSize
    self.name = name

    nn.Module.__init__(self)
    baseModel.__init__(self, None, self.name) # Model not passed into
        baseModel class to avoid reaching the maximum recursion depth.

    # Create a list to hold all layers
    layers = []
    # Define the layer sizes: start with inputSize, followed by your hidden
        layer sizes
    layer_sizes = [self.inputSize] + self.hiddenSizes

    # Loop to create each hidden layer with a ReLU activation
    for i in range(len(layer_sizes) - 1):
      layers.append(nn.Linear(layer_sizes[i], layer_sizes[i + 1]))
      layers.append(nn.Sigmoid())

    # Add the output layer (we don't usually follow the output layer with an
        activation like ReLU
    # if you're performing classification with CrossEntropyLoss, as it expects
        raw logits)
    layers.append(nn.Linear(hiddenSizes[-1], self.outputSize))
```

```
    # Use nn.Sequential to combine all the layers into a single network
    self.network = nn.Sequential(*layers)

  def forward(self, x):
    # The forward method defines how the data passes through the network layers
    return self.network(x)

  def predict(self, Xtest):
    Xtest = torch.tensor(Xtest.values, dtype=torch.float32)
    self.eval() # setting model to evaluation mode
    with torch.no_grad():
      outputs = self(Xtest)
      _, preds = torch.max(outputs, 1)

    preds = preds.numpy()
    print(type(preds))
    #predsdf = pd.DataFrame(preds, columns=["diagnosis"])
    return preds

  def dnnTrain(self, Xtrain, Ytrain, noEpochs = 5,  = 0.001):
    self.criterion = nn.CrossEntropyLoss()
    self.optimizer = optim.RMSprop(self.parameters(), lr=)

    for epoch in range(noEpochs): # Training for 5 epochs
      # Simulated dummy data (replace these with actual data during
          implementation)
      inputs = torch.tensor(Xtrain.values, dtype=torch.float32)
      labels = torch.tensor(Ytrain.values, dtype=torch.long)

      # Zero the gradients from the previous iteration
      self.optimizer.zero_grad()

      # Forward pass: Compute predicted outputs by passing inputs to the model
      outputs = self(inputs)

      # Calculate the loss
      loss = self.criterion(outputs, labels)

      # Backward pass: Compute gradient of the loss with respect to model
          parameters
      loss.backward()

      # Update parameters
      self.optimizer.step()

      # Print the loss for each epoch to track progress
      print(f"Epoch {epoch + 1}, Loss: {loss.item():.4f}")
    return [True, None, self]
```